

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems

COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO5: *Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency*

Assessment Tool Weightage: 35%

Dr VIMAL V R

QUESTIONS: 5

Total Marks:25

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

PSEUDOCODE WRITING TASK

S.No.	Questions	Mark
1	A system is at risk of disk overflow due to continuously increasing log files. Analyze the situation and develop a file system space management algorithm, using pseudocode, to monitor disk usage and automatically delete or rotate log files whenever disk utilization exceeds 80%. Pseudocode:BEGIN SET threshold = 80% usage = GET_DISK_USAGE() IF usage > threshold THEN IDENTIFY log_files SORTED BY size DESC FOR each file IN log_files DO DELETE or COMPRESS file usage = GET_DISK_USAGE() IF usage <= threshold THEN BREAK ENDIF ENDFOR ENDIF END	5
2	Improve CPU efficiency in a system by designing a priority-based scheduling algorithm. Develop and present the corresponding pseudocode to demonstrate how processes are selected and executed based on their priority levels. Pseudocode:BEGIN INPUT process_list SORT process_list BY priority (highest first) WHILE process_list NOT EMPTY DO SELECT process WITH highest priority ALLOCATE CPU time EXECUTE process IF process NOT completed THEN REDUCE priority slightly ADD back to process_list ENDIF ENDWHILE END	5
3	Efficiently manage and distribute RAM resources among multiple virtual machines in a system to ensure balanced performance and optimal utilization. Pseudocode:BEGIN total_RAM = 16GB host_reserved = 4GB available_RAM = total_RAM - host_reserved INPUT number_of_VMs min_RAM = 2GB FOR i = 1 TO number_of_VMs DO	5

	<pre> IF available_RAM >= min_RAM THEN ALLOCATE min_RAM to VM[i] available_RAM = available_RAM - min_RAM ELSE PRINT "Insufficient memory" ENDIF </pre>	
4	Select the most appropriate hypervisor for a given system environment by analyzing the specified requirements and constraints, and justify your decision to ensure optimal performance and efficient resource utilization. Pseudocode: BEGIN <pre> INPUT performance_required, budget, hardware_access IF performance_required = HIGH AND hardware_access = TRUE THEN SELECT Type_1_Hypervisor ELSE IF budget = LOW THEN SELECT Type_2_Hypervisor ELSE SELECT Type_1_Hypervisor ENDIF PRINT selected_hypervisor END </pre>	5
5	Manage and regulate CPU and memory usage of applications in a system by analyzing the scenario and implementing suitable resource-limiting mechanisms to ensure balanced and efficient system performance. Pseudocode: BEGIN <pre> INPUT application_list FOR each app IN application_list DO CREATE cgroup for app SET CPU limit SET memory limit ASSIGN process to cgroup ENDFOR END </pre>	5

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent	Mostly clear with minor	Difficult to follow in parts	Poorly structured and

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
		format, easy to follow	issues		unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANSWERS:

1. Disk Space Management

Aim:

To monitor disk usage and automatically compress or delete log files when disk utilization exceeds 80%.

Pseudocode:

```
BEGIN
    SET threshold = 80%
    usage = GET_DISK_USAGE()
    IF usage > threshold THEN
        IDENTIFY log files
        SORT log files BY size DESCENDING
        FOR each file IN log files DO
            COMPRESS or DELETE file
            usage = GET_DISK_USAGE()
            IF usage <= threshold THEN
                BREAK
            ENDIF
        ENDFOR
    ENDIF
    PRINT current disk usage
END
```

C Program:

```
#include <stdio.h>

int main() {
    int usage = 85;
```

```
int threshold = 80;
printf("Initial disk usage: %d%%\n", usage);
if (usage > threshold) {
    printf("Disk usage exceeded 80%%\n");
    printf("Compressing/Deleting log files...\n");
    usage = 75;
    printf("Log files managed successfully.\n");
}
printf("Final disk usage: %d%%\n", usage);
return 0;
}
```

Output:

Initial disk usage: 85%
Disk usage exceeded 80%
Compressing/Deleting log files...
Log files managed successfully.
Final disk usage: 75%

2. Priority-Based CPU Scheduling

Aim:

To improve CPU efficiency by selecting and executing processes according to their priority levels.

Pseudocode:

```
BEGIN
    INPUT process_list
    SORT process_list BY priority (highest first)
    WHILE process_list IS NOT EMPTY DO
        SELECT process WITH highest priority
        ALLOCATE CPU
        EXECUTE process
        IF process IS completed THEN
            REMOVE process
        ELSE
            REDUCE priority slightly
            ADD process BACK
        ENDIF
    ENDWHILE
    PRINT "All processes completed"
END
```

C Program:

```
#include <stdio.h>

int main() {
    int process[3] = {1, 2, 3};
    int priority[3] = {2, 5, 3};
    int i, j, temp;
```

```

for (i = 0; i < 3; i++) {
    for (j = i + 1; j < 3; j++) {
        if (priority[i] < priority[j]) {
            temp = priority[i];
            priority[i] = priority[j];
            priority[j] = temp;
            temp = process[i];
            process[i] = process[j];
            process[j] = temp;
        }
    }
}
printf("Processes executed by priority:\n");
for (i = 0; i < 3; i++) {
    printf("P%d - Priority %d - Executed\n",
        process[i], priority[i]);
}
return 0;
}

```

Output:

```

Processes executed by priority:
P2 - Priority 5 - Executed
P3 - Priority 3 - Executed
P1 - Priority 2 - Executed

```


3. RAM Allocation for Virtual Machines

Aim:

To efficiently allocate RAM among multiple virtual machines while maintaining balanced memory utilization.

Pseudocode:

```
BEGIN
    SET total_RAM = 16 GB
    SET host_reserved = 4 GB
    available_RAM = total_RAM - host_reserved
    INPUT number_of_VMs
    SET min_RAM = 2 GB
    FOR i = 1 TO number_of_VMs DO
        IF available_RAM >= min_RAM THEN
            ALLOCATE min_RAM TO VM[i]
            available_RAM = available_RAM - min_RAM
        ELSE
            PRINT "Insufficient memory"
        ENDIF
    ENDFOR
    PRINT remaining RAM
END
```

C Program:

```
#include <stdio.h>

int main() {
    int totalRAM = 16;
    int reserved = 4;
    int available = totalRAM - reserved;
```

```

int minRAM = 2;
int n, i;
printf("Total RAM: %d GB\n", totalRAM);
printf("Host Reserved: %d GB\n", reserved);
printf("Available RAM: %d GB\n", available);
printf("Enter number of VMs: ");
scanf("%d", &n);
for (i = 1; i <= n; i++) {
    if (available >= minRAM) {
        printf("VM%d allocated %d GB RAM\n", i, minRAM);
        available -= minRAM;
    } else {
        printf("VM%d: Insufficient memory\n", i);
    }
}
printf("Remaining RAM: %d GB\n", available);
return 0;
}

```

Output:

```

Total RAM: 16 GB
Host Reserved: 4 GB
Available RAM: 12 GB
Enter number of VMs: 5
VM1 allocated 2 GB RAM
VM2 allocated 2 GB RAM
VM3 allocated 2 GB RAM
VM4 allocated 2 GB RAM
VM5 allocated 2 GB RAM
Remaining RAM: 2 GB

```

4. Hypervisor Selection

Aim:

To select the most suitable hypervisor based on performance, budget, and hardware access requirements.

Pseudocode:

```
BEGIN
    INPUT performance_required
    INPUT budget
    INPUT hardware_access
    IF performance_required = HIGH AND hardware_access = TRUE
    THEN
        SELECT Type_1_Hypervisor
    ELSE IF budget = LOW THEN
        SELECT Type_2_Hypervisor
    ELSE
        SELECT Type_1_Hypervisor
    ENDIF
    PRINT selected_hypervisor
END
```

C Program:

```
#include <stdio.h>

int main() {
    int performance, budget, hardware;
    printf("Enter performance required (1=High, 0=Low): ");
    scanf("%d", &performance);
    printf("Enter budget (1=High, 0=Low): ");
    scanf("%d", &budget);
```

```
printf("Hardware access available (1=Yes, 0=No): ");
scanf("%d", &hardware);
if (performance == 1 && hardware == 1)
    printf("Selected Hypervisor: Type-1 Hypervisor\n");
else if (budget == 0)
    printf("Selected Hypervisor: Type-2 Hypervisor\n");
else
    printf("Selected Hypervisor: Type-1 Hypervisor\n");
return 0;
}
```

Output:

Enter performance required (1=High, 0=Low): 1

Enter budget (1=High, 0=Low): 1

Hardware access available (1=Yes, 0=No): 1

Selected Hypervisor: Type-1 Hypervisor

5. CPU and Memory Resource Limiting

Aim:

To control CPU and memory usage of applications by assigning suitable resource limits.

Pseudocode:

```
BEGIN
    INPUT application_list
    FOR each app IN application_list DO
        CREATE resource_group for app
        SET CPU limit
        SET memory limit
        ASSIGN app TO resource_group
    ENDFOR
    MONITOR CPU and memory usage
    IF usage exceeds limit THEN
        RESTRICT resource usage
    ENDIF
    PRINT "Resource limits applied"
END
```

C Program:

```
#include <stdio.h>

int main() {
    int n, i;
    char app[20];
    printf("Enter number of applications: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
```

```
    printf("Enter application name: ");
    scanf("%s", app);
    printf("Application: %s\n", app);
    printf("CPU Limit: 50%%\n");
    printf("Memory Limit: 2 GB\n");
    printf("Resource limits applied successfully.\n\n");
}
return 0;
}
```

Output:

```
Enter number of applications: 2
Enter application name: Chrome
Application: Chrome
CPU Limit: 50%
Memory Limit: 2 GB
Resource limits applied successfully.
```

```
Enter application name: VSCode
Application: VSCode
CPU Limit: 50%
Memory Limit: 2 GB
Resource limits applied successfully.
```